

Labo 14 – Verslag JavaScript deel 2

opleidingsonderdeel **Web Development I**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Tibo Dewanckele

docent: **Pim Debaere**

academiejaar **2025–2026**

Inhoud

Opdracht codecademy	3
Opdracht: arrays	4
Opdracht: dialoogvensters	5
Opdracht: innerHTML	6
Opdracht: debuggen	6
Opdracht: innerHTML aanpassen	9
Opdracht: kopieer	9
Opdracht: substring.....	11

Opdracht codecademy



Arrays



Loops



Methodes om te kennen

array.push(element)

Dit zal het meegegeven element op het einde van de array toevoegen.

array.pop()

Dit zal het laatste element uit de array verwijderen.

array.shift()

Dit zal het eerste element uit de array verwijderen.

array.unshift(element)

Dit zal het meegegeven element vooraan in de array toevoegen.

array.slice(eerste_element, stop_element)

Dit zal een deel van de array nemen startende vanaf het eerste element en zal stoppen voor het stop_element.

array.join(string)

Dit zal alle elementen van de array samenvoegen in een string waarbij dat er tussen de elementen de meegegeven string zit.

array.indexOf(element,index)

Dit zoekt het element in de array en geeft de laagste index terug waar het voorkomt.

array.lastIndexOf(element,index)

Dit zoekt het element in de array en geeft de hoogste index terug waar het voorkomt.

→ Bij zowel **indexOf** als **lastIndexOf** zal als het element niet wordt teruggevonden **-1** teruggegeven worden.

→ Bij zowel **indexOf** als **lastIndexOf** is de **parameter index optioneel** en geeft die aan op welke index je moet starten in de array.

→ Om elementen te vergelijken gebruiken we **===**

Opdracht: arrays

Eerst maken we een array aan.

```
const familieleden = ['Max', 'Tibo', 'Lewis', 'Romeo', 'Julia'];
```

We willen naar de console schrijven hoeveel elementen de array bevat.

Via **length** kunnen we hier weten hoeveel erin zitten.

```
console.log(familieleden.length);
```

5

Dan willen we het 1^{ste}, 3^{de} en 5^{de} element van de array naar de console wordt geschreven.

Via een for loop kunnen we de array overlopen en met stappen van 2 door de array gaan.

Omdat een array bij 0 start staat het 1^{ste} element op index 0 enz.

```
for (let i = 0; i < familieleden.length; i+=2) {  
  console.log(familieleden[i]);  
}
```

Max

Lewis

Julia

We willen dan ook nog aan de gebruiker vragen of ze nog een familielid willen toevoegen aan de array. Dit kan gedaan worden via **prompt()**. We schrijven zelf een methode om dit uit te voeren genaamd VoegNaamToe.

We moeten gebruik maken van pass-by reference wat betekent dat we de array meegeven met de methode en dan in de methode aanpassingen aan de array doorvoeren. Via push() wordt deze toegevoegd aan de array.

```
const VoegNaamToe = (familieleden) => {  
  familieleden.push(prompt("Wat is de naam van uw familielid?"));  
}
```

localhost:63342 meldt het volgende

Wat is de naam van uw familielid?

|

OK

Annuleren

```
▶ (6) ['Max', 'Tibo', 'Lewis', 'Romeo', 'Julia', 'justin']
```

We roepen deze dan aan in de setup() en daar doen we dan ook nog eens een console.log van de array.

Omdat we dan ook nog eens alle leden in 1 string willen zien gebruiken we **join()**.

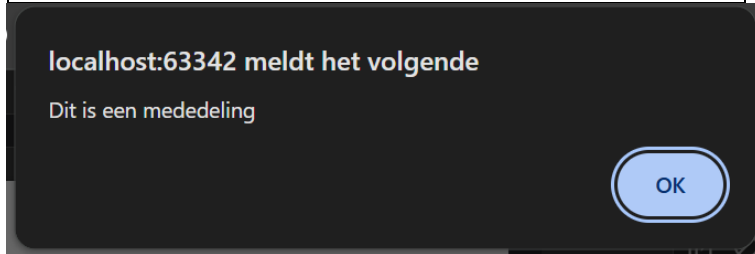
```
console.log(familieleden.join(' '));
```

Max Tibo Lewis Romeo Julia justin

Opdracht: dialogovensters

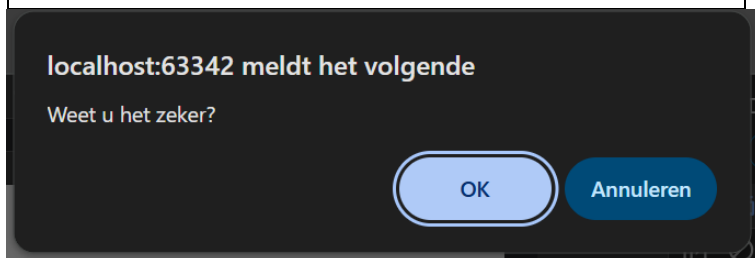
Alert

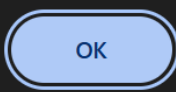
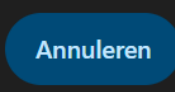
```
window.alert("Dit is een mededeling");
```



Confirm

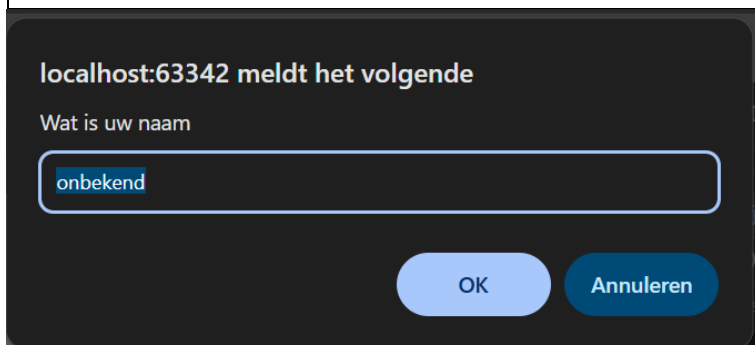
```
window.confirm("Weet u het zeker?")
```

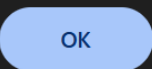
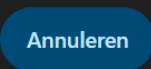


		
Output console	true	false

Prompt

```
prompt("Wat is uw naam", "onbekend")
```



		
Output console	Ingetypte tekst	null

Opdracht: innerHTML

Als je onderstaande regel in je body schrijft dan krijg je volgende output op de website.

```
<p id="txtOutput">Hello world!</p>
```

Hello world!

Door nu volgende regel toe te voegen op het einde van je body zodat eerst alles wordt ingeladen en dan het script wordt uitgevoerd.

```
<script src="scripts/script.js"></script>
```

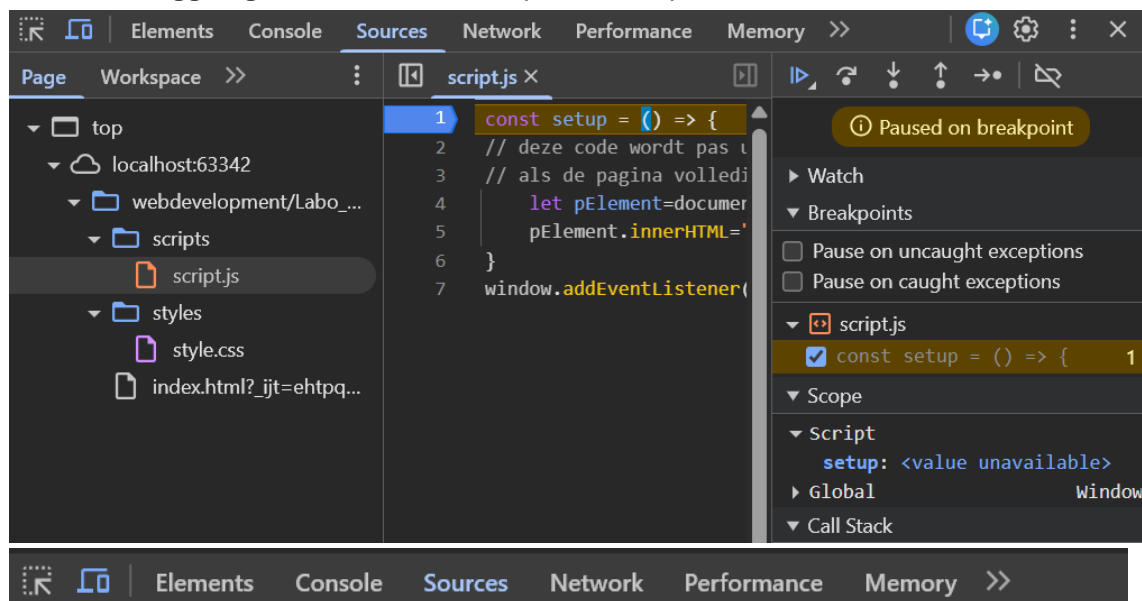
Via onderstaande regels code kunnen we nu ook de inhoud van de website aanpassen. Het haalt eerst het HTML-element op met id txtOutput via **document.getElementById**. Dan veranderen we de inhoud van dat element via **innerHTML**.

```
let pElement=document.getElementById("txtOutput");  
pElement.innerHTML="Welkom!";
```

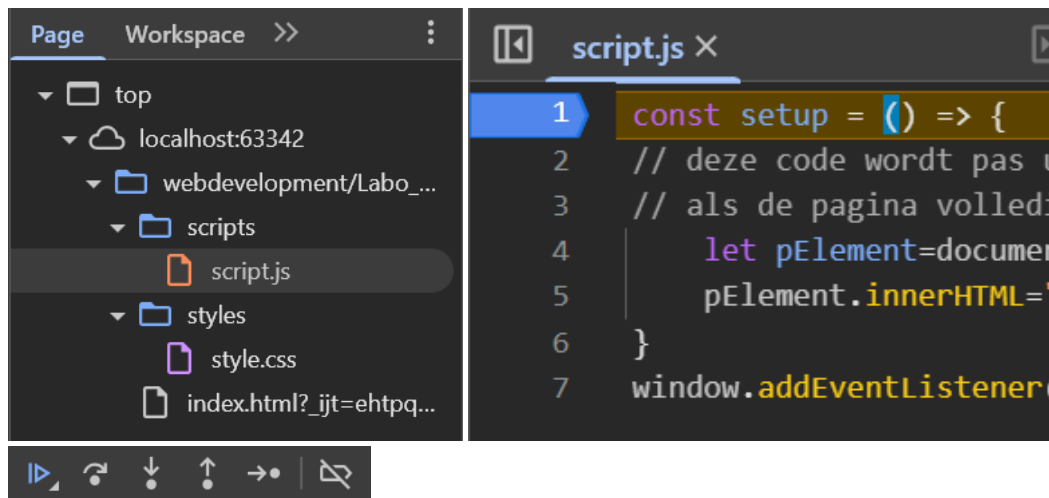
Welkom!

Opdracht: debuggen

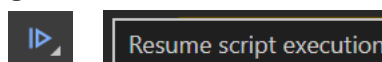
Om te debuggen gaan we naar developer tools op het tablad **Sources**.



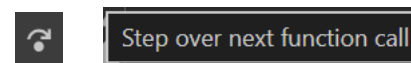
Dan kijk je naar **Page** en navigeer je naar het bestand dat je wilt debuggen.
Dan krijg je in het venstertje ernaast de inhoud van het geselecteerde bestand.
Door op nummer van de lijn te drukken zal hij op dat moment stoppen zodat je kan debuggen (deze lijn wordt dan gemarkeerd).



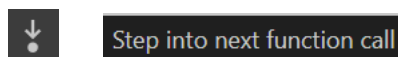
Via bovenstaande knoppen kan je verder stappen en dus stap per stap bekijken wat er gebeurt.



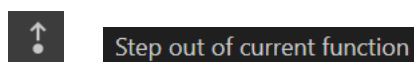
Deze stopt de debugging mode en hervat gewoon alles.



Deze knop gaat gewoon over de methode in zijn totaal (slaat hem gewoon over) en gaat door naar de regel achter de methode dat je op dit moment op staat.



Als je op een methode stapt dan gaat deze naar de methode zelf en begint hij deze te overlopen anders regel per regel.



Als je in een methode zit en je drukt op deze knop dan zal deze er direct uitstappen en de regels die deze methode volgen beginnen bekijken.



Deze zal gewoon regel per regel bekijken en zal niet dieper in een methode gaan.

```
script.js X
1  const setup = () => {
2    // deze code wordt pas u
3    // als de pagina volledi
4    let pElement=document
5    pElement.innerHTML='
6  }
7  window.addEventListener(
```

Als we debuggen op voorgaande oefening (innerHTML) en de breakpoint op de eerste regel van het script zetten dan zal het script nog niet uitgevoerd zijn.

```
Hello world!
Paused in debugger
```

Het resultaat is dat de output op het scherm nog steeds *Hello world!* is.

Alles is nu gepauzeerd en het script wordt nog niet uitgevoerd.

In de DOM-tree is dit nu ook nog niet aangepast

```
<!DOCTYPE html>
<html lang="en"> scroll
  <head> ... </head>
  <body>
... <p id="txtOutput">Hello world!</p> == $0
    <script src="scripts/script.js"></script>
  </body>
</html>
```

Als we verder stappen via die knoppen, dan voeren we regel per regel code van het script uit. Als we dit doen tot dat de regels uitgevoerd worden om dit te veranderen naar *Welkom!*

```
Welkom!
Paused in debugger
```

Op dat moment is het ook in de DOM-tree veranderd.

```
<!DOCTYPE html>
<html lang="en"> scroll
  <head> ... </head>
  <body>
.. <p id="txtOutput">Welkom!</p> == $0
    <script src="scripts/script.js"></script>
  <script> ... </script>
  </body>
</html>
```


Opdracht: innerHTML aanpassen

We voegen een knop toe waarop er *Wijzig* op staat, dat doen we via de **input** tag. Het **type** is dan **button** en via **value** geven we mee wat de tekst op deze knop is. We kennen ook een **id** aan zodat we dit element kunnen ophalen in het script.

```
<p id="txtOutput">Hello world!</p>
<input type="button" id="WijzigButton" value="Wijzig">
<script src="scripts/script.js"></script>
```

We willen dat het script niet wordt uitgevoerd tot dat we op de knop drukken.

In de setup halen we dus nu het element op met id *txtOutput*.

Nu kunnen we een **addEventListener** toevoegen om te kunnen weten wanneer er iets wordt gedaan met dat HTML-element.

We geven als parameters mee dat de activiteit een klik moet zijn op de knop via **“click”**. Dan geef je mee welke functie moet worden uitgevoerd als er op wordt gedrukt.

```
let wijzigButton = document.getElementById("WijzigButton");
wijzigButton.addEventListener("click", wijzig);
```

Omdat we de tekst willen aanpassen op het moment dat er op de knop wordt geklikt dus de regels code om dit aan te passen zetten we dan in de functie die is gekoppeld bij **addEventListener**.

```
const wijzig = () => {
  let pElement=document.getElementById("txtOutput");
  pElement.innerHTML="Welkom!";
}
```

Opdracht: kopieer

We voegen een **inputveld** en een **knop (button)** op de website toe met de tekst Kopieer.

```
<input id="txtInput" type="tekst" />
<input id="btnKopieer" type="button" value="Kopieer" />
```

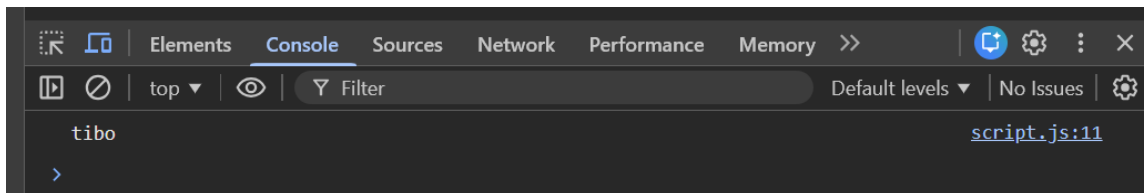
We willen dus opnieuw controleren wanneer er op de knop wordt gedrukt.

We koppelen dan aan die knop de functie *kopieer*.

```
let btnKopieer = document.getElementById("btnKopieer");
btnKopieer.addEventListener("click", kopieer);
```

We halen dan de waarde van het inputveld op door eerst het HTML-element op te halen. Daarna sla je de value ervan op via variabelenaam **.value** . Dan loggen we dit in de console.

```
Const kopieer = () => {  
  let txtInput = document.getElementById("txtInput");  
  let tekst = txtInput.value;  
  console.log(tekst);  
}
```

We willen nu dat de tekst dat we kopiëren uit het inputveld in een p tag op de website komt. Deze komt gewoon in de body.

```
<p id="txtOutput">geen output</p>
```

We halen nog steeds de waarde van het inputveld op via dezelfde manier als hiervoor. We slaan de value ervan op in tekst. Dan halen het HTML element op met het id txtOutput die dan de p tag is die we hebben toegevoegd. Dan kennen we de waarde van het inputveld toe aan txtOutput en wordt de inhoud van de p tag hetzelfde als de inhoud van het inputveld.

```
const kopieer = () => {  
  let txtInput = document.getElementById("txtInput");  
  let tekst = txtInput.value;  
  let txtOutput = document.getElementById("txtOutput");  
  txtOutput.innerHTML = tekst;  
}
```


tibo

Opdracht: substring

We willen dus op de website een woord kunnen ingeven en dan de startwaarde en de eindwaarde voor de methode Substring kunnen ingeven.

Op het moment dat er op Substring wordt gedrukt moet het uitgevoerd worden en worden getoond op de website.

"" (,) = pel

Om al deze inputvelden en knoppen te hebben hebben we volgende HTML nodig hebben. Omdat er nog tekens rond de velden op dezelfde lijn staan heb ik gekozen om de inputvelden in een p tag te steken.

We kennen id's toe aan deze inputvelden en de span op het einde voor de uitvoer. Zo kunnen we ze in het script ophalen.

```
<p>
  <input type="text" id="txtInput">
  <input type="button" id="btnSubstring" value="Substring">
  (<input type="number" id="startgetal">,
  <input type="number" id="eindgetal">)
  = <span id="txtOutput">(geen output)</span>
</p>
```

Nu willen we dat de substring methode wordt uitgevoerd als er op de knop Substring wordt gedrukt. We halen dus het HTML element op via getElementById.

Via addEventListener kunnen we een actie koppelen aan een klik op de knop Substring.

We gaan zelf een methode aanmaken waarin dat de substring methode uiteindelijk wordt uitgevoerd.

```
const setup = () => {
  let btnSubstring = document.getElementById("btnSubstring");
  btnSubstring.addEventListener("click", substring);
}
```

We beginnen dus met het aanmaken van deze methode op volgende manier.

```
const substring = () => {}
```

Binnen deze methode willen we de HTML elementen van de inputvelden ophalen dus wordt volgende binnen de methode gedaan.

```
let txtInput = document.getElementById("txtInput");
const startgetal = document.getElementById("startgetal");
const eindgetal = document.getElementById("eindgetal");
```

We zullen in dit geval altijd met de waarde van de HTML elementen werken dus gaan we deze opslaan in aparte variabelen.

```
const tekst = txtInput.value;
const startWaarde = startgetal.value;
const eindWaarde = eindgetal.value;
```

Om te zorgen dat er correcte waarden worden opgegeven als parameters in de functie substring voegen we een if toe. Deze controleert of startGetal groter of gelijk is aan 0 en het eindGetal kleiner of gelijk is aan de lengte van het ingegeven woord.

```
if(tekst.length >= eindWaarde && startWaarde >= 0){
    let txtOutput = document.getElementById("txtOutput");
    txtOutput.innerHTML = tekst.substring(startWaarde,eindWaarde);
}
```

Binnen de if halen we het HTML element op voor de output en aan de innerHTML van het opgehaalde HTML element kennen we dan het resultaat van de **substring** op tekst toe.